Linux Common Analysis & Observability Tools



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



Pseudo Filesystems



Pseudo Filesystems

- Some virtual filesystems are exposed by the kernel and provide a lot of information on the system.
- *procfs* contains information about processes and system information.
  - Mounted on `/proc`
  - Often parsed by tools to display raw data in a more user-friendly way.
- *sysfs* provides information about hardware/logical devices, association between devices and drivers.
  - Mounted on `/sys`
- *debugfs* exposes information related to debug.
  - Typically mounted on `/sys/kernel/debug/`
  - `mount -t debugfs none /sys/kernel/debug`



procfs

- *procfs* exposes information about processes and system (man 5 proc).
  - `/proc/cpuinfo` CPU information.
  - `/proc/meminfo` memory information (used, free, total, etc).
  - `/proc/sys/` contains system parameters that can be tuned. The list of parameters that can be modified is available at admin-guide/sysctl/index
  - `/proc/interrupts`: interrupt count per CPU for each interrupt in use
    - We also have one entry per interrupt in `/proc/irq` for specific configuration/status for each interrupt line
  - `/proc/<pid>/` process related information
    - `/proc/<pid>/status` process basic information
    - `/proc/<pid>/maps` process memory mappings
    - `/proc/<pid>/fd` file descriptors of the process
    - `/proc/<pid>/task` descriptors of threads belonging to the process
  - `/proc/self/` will refer to the process used to access the file
- A list of all available *procfs* file and their content is described at filesystems/proc and man 5 proc



sysfs

- *sysfs* filesystem exposes information about various kernel subsystems, hardware devices and association with drivers (man 5 sysfs).
- `/sys/bus` contains one directory per bus types
  - `/sys/bus/<bus>/drivers` shows all drivers attached to a bus
  - `/sys/bus/<bus>/devices` shows all devices sitting on a bus
  - Symlinks inside those directories allows inspecting the relationship between devices and drivers
- `/sys/class` contains a tree of class devices registered by drivers
- `/sys/kernel` contains interesting files for kernel debugging:
  - `irq` with information about interrupts (mapping, count, etc).
  - `tracing` for tracing control.
- admin-guide/abi-stable



debugfs

- *debugfs* is a simple RAM-based filesystem which exposes debugging information.
- Used by some subsystems (*clk*, *block*, *dma*, *gpio*, etc) to expose debugging information related to the internals.
- Usually mounted on `/sys/kernel/debug`
  - Dynamic debug features exposed through `/sys/kernel/debug/dynamic_debug` (also exposed in `proc`)
  - Clock tree exposed through `/sys/kernel/debug/clk/clk_summary`.



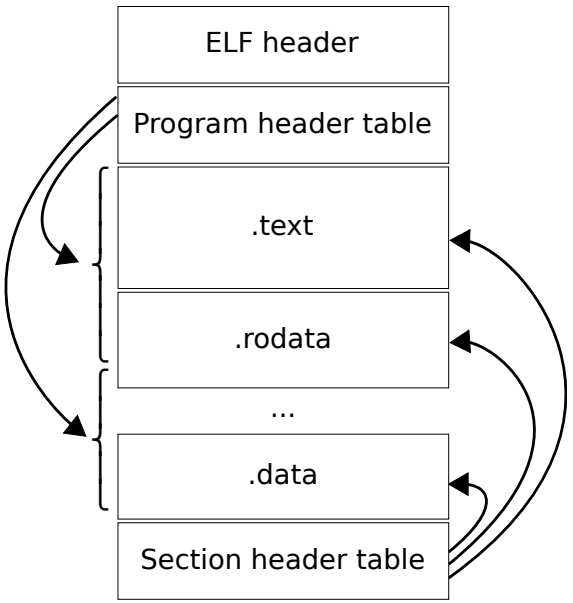
ELF file analysis



ELF files

Executable and **L**inkable **F**ormat

- File starting with a header which holds binary structures defining the file
- Collection of segments and sections that contain data
  - `.text` section: Code
  - `.data` section: Data
  - `.rodata` section: Read-only Data
  - `.debug_info` section: Contains debugging information
- Sections are part of a segment which can be loadable in memory
- Same format for all architectures supported by the kernel and also `vmlinux` format
  - Also used by a lot of other operating systems as the standard executable file format





binutils for ELF analysis

- The binutils are used to deal with binary files, either object files or executables.
  - Includes `ld`, `as` and other useful tools.
- *readelf* displays information about ELF files (header, section, segments, etc).
- *objdump* allows to display information and disassemble ELF files.
- *objcopy* can convert ELF files or extract/translate some parts of it.
- *nm* displays the list of symbols embedded in ELF files.
- *addr2line* finds the source code line/file pair from an address using an ELF file with debug information



binutils example (1/2)

- Finding the address of `ksys_read()` kernel function using *nm*:

```
$ nm vmlinux | grep ksys_read
c02c7040 T ksys_read
```

- Using *addr2line* to match a kernel OOPS address or a symbol name with source code:

```
$ addr2line -s -f -e vmlinux ffffffff8145a8b0
queue_wc_show
blk-sysfs.c:516
```



binutils example (2/2)

- Display an ELF header with *readelf*:

```
$ readelf -h binary
ELF Header:
Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
Class:                             ELF64
Data:                              2's complement, little endian
Version:                           1 (current)
OS/ABI:                            UNIX - System V
ABI Version:                       0
Type:                              DYN (Position-Independent Executable file)
Machine:                           Advanced Micro Devices X86-64
...
```

- Convert an ELF file to a flat binary file using *objcopy*:

```
$ objcopy -O binary file.elf file.bin
```



ldd

- In order to display the shared libraries used by an ELF binary, one can use *ldd* (Generally packaged with C library. See man 1 ldd).
- *ldd* will list all the libraries that were used at link time.
  - Libraries that are loaded at runtime using `dlopen()` are not displayed.

```
$ ldd /usr/bin/bash
linux-vdso.so.1 (0x00007ffdf3fc6000)
libreadline.so.8 => /usr/lib/libreadline.so.8 (0x00007fa2d2aef000)
libc.so.6 => /usr/lib/libc.so.6 (0x00007fa2d2905000)
libncursesw.so.6 => /usr/lib/libncursesw.so.6 (0x00007fa2d288e000)
/lib64/ld-linux-x86-64.so.2 => /usr/lib64/ld-linux-x86-64.so.2 (0x00007fa2d2c88000)
```



Monitoring tools



Monitoring Tools

- Lots of monitoring tools on Linux to allow monitoring various part of the system.
- Most of the time, these are CLI interactive programs.
  - Processes with *ps*, *top*, *htop*, etc
  - Memory with *free*, *vmstat*
  - Networking
- Almost all these tools rely on the *sysfs* or *procfs* filesystem to obtain the processes, memory and system information but will display them in a more human-readable way.
  - Networking tools use a netlink interface with the networking subsystem of the kernel.



Process and CPU monitoring tools



Processes with *ps*

- The *ps* command allows to display a snapshot of active processes and their associated information (man 1 ps)
  - Lists both user processes and kernel threads.
  - Displays PID, CPU usage, memory usage, uptime, etc.
  - Uses `/proc/<pid>/` directory to obtain process information.
  - Almost always present on embedded platforms (provided by *BusyBox*).
- By default, displays only the current user/current tty processes, but output is highly customizable:
  - `aux`/`-e`: show all processes
  - `-L`: show threads
  - `-p`: target a specific process
  - `-o`: select output columns to display
- Useful for scripting and parsing since its output is static.



ps example

- Display all processes in a friendly way:

```
$ ps aux
USER    PID %CPU %MEM    VSZ   RSS TTY STAT START  TIME COMMAND
root      1  0.0  0.0 168864 12800 ?   Ss   09:08  0:00 /sbin/init
root      2  0.0  0.0      0     0 ?   S    09:08  0:00 [kthreadd]
root      3  0.0  0.0      0     0 ?   I<   09:08  0:00 [rcu_gp]
root      4  0.0  0.0      0     0 ?   I<   09:08  0:00 [rcu_par_gp]
root      5  0.0  0.0      0     0 ?   I<   09:08  0:00 [netns]
[...]
root    914  0.0  0.0 396216 16220 ?   Ssl  09:08  0:04 /usr/libexec/udisks2/udisksd
avahi   929  0.0  0.0   8728   412 ?   S    09:08  0:00 avahi-daemon: chroot helper
root    956  0.0  0.1 260304 19024 ?   Ssl  09:08  0:02 /usr/sbin/NetworkManager [...]
root    960  0.0  0.0  17040  5704 ?   Ss   09:08  0:00 /sbin/wpa_supplicant -u [...]
root    962  0.0  0.0 317644 11896 ?   Ssl  09:08  0:00 /usr/sbin/ModemManager
vnstat  987  0.0  0.0   5516  3696 ?   Ss   09:08  0:00 /usr/sbin/vnstatd -n
```



Processes with *top*

- *top* command output information similar to *ps* but dynamic and interactive ().
  - Also almost always present on embedded platforms (provided by *BusyBox*)

```
$ top
top - 18:38:11 up  9:29,  1 user,  load average: 2.84, 2.74, 2.02
Tasks: 371 total,   1 running, 370 sleeping,   0 stopped,   0 zombie
%Cpu(s):  5.8 us,  2.1 sy,  0.0 ni, 77.4 id, 14.7 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :  15947.6 total,   1476.9 free,   7685.7 used,   6784.9 buff/cache
MiB Swap:  15259.0 total,  15238.7 free,     20.2 used.   7742.3 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   2988 cleger    20   0 5184816   1.2g 430244 S  26.7   7.9  60:24.27 firefox-esr
   4326 cleger    20   0   16.4g 208104  81504 S  26.7   1.3   9:27.33 code
    909 root     -51   0       0      0      0 S  13.3   0.0  15:12.15 irq/104-nvidia
  41704 cleger    20   0   38.4g 373744 116984 S  13.3   2.3  13:25.76 code
  91926 cleger    20   0 2514784 145360  95144 S  13.3   0.9   1:29.85 Web Content
```



mpstat

- *mpstat* displays Multiprocessor statistics (man 1 mpstat).
- Useful to detect unbalanced CPU workloads, bad IRQ affinity, etc.

```
$ mpstat -P ALL
Linux 6.0.0-1-amd64 (fixe)      19/10/2022      _x86_64_        (4 CPU)

17:02:50     CPU    %usr   %nice    %sys %iowait    %irq   %soft  %steal  %guest  %gnice   %idle
17:02:50     all    6,77    0,00    2,09   11,67    0,00    0,06    0,00    0,00    0,00   79,40
17:02:50       0    6,88    0,00    1,93    8,22    0,00    0,13    0,00    0,00    0,00   82,84
17:02:50       1    4,91    0,00    1,50    8,91    0,00    0,03    0,00    0,00    0,00   84,64
17:02:50       2    6,96    0,00    1,74    7,23    0,00    0,01    0,00    0,00    0,00   84,06
17:02:50       3    9,32    0,00    2,80   54,67    0,00    0,00    0,00    0,00    0,00   33,20
```



Memory monitoring tools



free

- *free* is a simple program that displays the amount of free and used memory in the system (man 1 free).
  - Useful to check if the system suffers from memory exhaustion
  - Uses `/proc/meminfo` to obtain memory information.

```
$ free -h
               total        used        free      shared  buff/cache   available
Mem:            15Gi       7.5Gi       1.4Gi       192Mi       6.6Gi       7.5Gi
Swap:           14Gi        20Mi        14Gi
```

- *A small `free` value does not mean that your system suffers from memory depletion! Linux considers any unused memory as "wasted" so it uses it for buffers and caches to optimize performance. See also `drop_caches` from man 5 proc to observe buffers/cache impact on free/available memory*



vmstat

- *vmstat* displays information about system virtual memory usage
- Can also display stats from processes, memory, paging, block IO, traps, disks and cpu activity (man 8 vmstat).
- Can be used to gather data at periodic interval using `vmstat <interval> <number>`

```
$ vmstat 1 6
procs -----------memory----------   ---swap--  -----io---- -system-- ------cpu-----
r  b   swpd   free   buff  cache     si   so    bi    bo    in   cs  us sy id wa st
3  0 253440 1237236 194936 9286980    3    6   186   540    134  157  3  5 82 10  0
```

- *Note: vmstat consider a kernel block to be 1024 bytes*



pmap

- `pmap` displays process mappings more easily than accessing `/proc/<pid>/maps` (man 1 pmap).

```
# pmap 2002
2002:   /usr/bin/dbus-daemon --session --address=systemd: --nofork --nopidfile --systemd-activation
--syslog-only
...
00007f3f958bb000     56K r---- libdbus-1.so.3.32.1
00007f3f958c9000    192K r-x-- libdbus-1.so.3.32.1
00007f3f958f9000     84K r---- libdbus-1.so.3.32.1
00007f3f9590e000      8K r---- libdbus-1.so.3.32.1
00007f3f95910000      4K rw--- libdbus-1.so.3.32.1
00007f3f95937000      8K rw---   [ anon ]
00007f3f95939000      8K r---- ld-linux-x86-64.so.2
00007f3f9593b000    152K r-x-- ld-linux-x86-64.so.2
00007f3f95961000     44K r---- ld-linux-x86-64.so.2
00007f3f9596c000      8K r---- ld-linux-x86-64.so.2
00007f3f9596e000      8K rw--- ld-linux-x86-64.so.2
00007ffe13857000    132K rw---   [ stack ]
00007ffe13934000     16K r----   [ anon ]
00007ffe13938000      8K r-x--   [ anon ]
 total            11088K
```



I/O monitoring tools



iostat

- *iostat* displays information about IOs per device on the system.
- Useful to see if a device is overloaded by IOs.

```
$ iostat
Linux 5.19.0-2-amd64 (fixe)     11/10/2022      _x86_64_        (12 CPU)

avg-cpu:  %user   %nice %system %iowait  %steal   %idle
           8,43    0,00    1,52    8,77    0,00   81,28

Device      tps  kB_read/s  kB_wrtn/s  kB_dscd/s  kB_read  kB_wrtn  kB_dscd
nvme0n1   55,89    1096,88     149,33       0,00  5117334   696668        0
sda        0,03       0,92       0,00       0,00     4308        0        0
sdb      104,42     274,55    2126,64       0,00  1280853  9921488        0
```



iotop

- *iotop* displays information about IOs much like *top* for each process.
- Useful to find applications generating too much I/O traffic.
  - Needs CONFIG_TASKSTATS=y, CONFIG_TASK_DELAY_ACCT=y and CONFIG_TASK_IO_ACCOUNTING=y to be enabled in the kernel.
  - Also needs to be enabled at runtime: `sysctl -w kernel.task_delayacct=1`

```
# iotop
Total DISK READ:        20.61 K/s | Total DISK WRITE:        51.52 K/s
Current DISK READ:      20.61 K/s | Current DISK WRITE:      24.04 K/s
    TID  PRIO  USER     DISK READ DISK WRITE>    COMMAND
    2629 be/4 cleger     20.61 K/s   44.65 K/s firefox-esr [Cache2 I/O]
    322 be/3 root        0.00 B/s    3.43 K/s [jbd2/nvme0n1p1-8]
  39055 be/4 cleger      0.00 B/s    3.43 K/s firefox-esr [DOMCacheThread]
      1 be/4 root        0.00 B/s    0.00 B/s init
      2 be/4 root        0.00 B/s    0.00 B/s [kthreadd]
      3 be/0 root        0.00 B/s    0.00 B/s [rcu_gp]
      4 be/0 root        0.00 B/s    0.00 B/s [rcu_par_gp]
      ...
```



Networking observability tools



ss

- *ss* shows the status of network sockets
  - IPv4 and IPv6, UDP, TCP, ICMP and UNIX domain sockets
- Replaces *netstat*, now obsolete
- Gets info from `/proc/net`
- Usage: `ss` by default shows connected sockets
  `ss -l` shows listening sockets
  `ss -a` shows both listening and connected sockets
  `ss -4/-6/-x` shows only IPv4, IPv6, or UNIX sockets
  `ss -t/-u` shows only TCP or UDP sockets
  `ss -p` shows process using each socket
  `ss -n` shows numeric addresses
  `ss -s` shows a summary of existing sockets
- See the ss manpage for all the options



ss example output

```
# ss
Netid State  Recv-Q Send-Q                Local Address:Port            Peer Address:Port  Process
u_dgr ESTAB  0      0                                 * 304840                     * 26673
u_str ESTAB  0      0       /run/dbus/system_bus_socket 42871                     * 26100
icmp6 UNCONN 0      0                                 *:ipv6-icmp                 *:*
udp   ESTAB  0      0          192.168.10.115%wlp0s20f3:bootpc         192.168.10.88:bootps
tcp   ESTAB  0      136                   172.16.0.1:41376          172.16.11.42:ssh
tcp   ESTAB  0      273                  192.168.1.77:55494         87.98.181.233:https
tcp   ESTAB  0      0                 [2a02:...:dbdc]:38466     [2001:...:9]:imap2
...
#
```



iftop

- *iftop* displays bandwidth usage on an interface by remote host
- Visualizes bandwidth using histograms
- `iftop -i eth0`

```
                 19.1Mb          38.1Mb          57.2Mb          76.3Mb          95.4Mb
└──────────────────┴───────────────┴───────────────┴───────────────┴───────────────┘
localhost.localnet                => ams.source.kernel.org          424Kb   352Kb   403Kb
[redacted]                        <=                                18.8Mb  15.6Mb  18.2Mb
localhost.localnet                => bootlin.com                      0b    113Kb  35.0Kb
[redacted]                        <=                                  0b    6.20Mb  1.84Mb

TX:             cum:   3.06MB   peak:    836Kb          rates:     424Kb   465Kb   439Kb
RX:                    146MB            44.3Mb                     18.8Mb  21.8Mb  20.1Mb
TOTAL:                 149MB            45.1Mb                     19.2Mb  22.3Mb  20.5Mb
```

- The output can be customized interactively
- See the iftop manpage for details



tcpdump

- *tcpdump* allows to capture network traffic and decode many protocols
- `tcpdump -i eth0`
- based on the *libpcap* library for packet capture
- It can also store captured packets to a file and read them back
  - In the *pcap* format or the newer *pcapng* format
  - `tcpdump -i eth0 -w capture.pcap`
  - `tcpdump -r capture.pcap`
- A BPF capture filter can be used to avoid capturing irrelevant packets
  - `tcpdump -i eth0 tcp and not port 22`



tcpdump example output

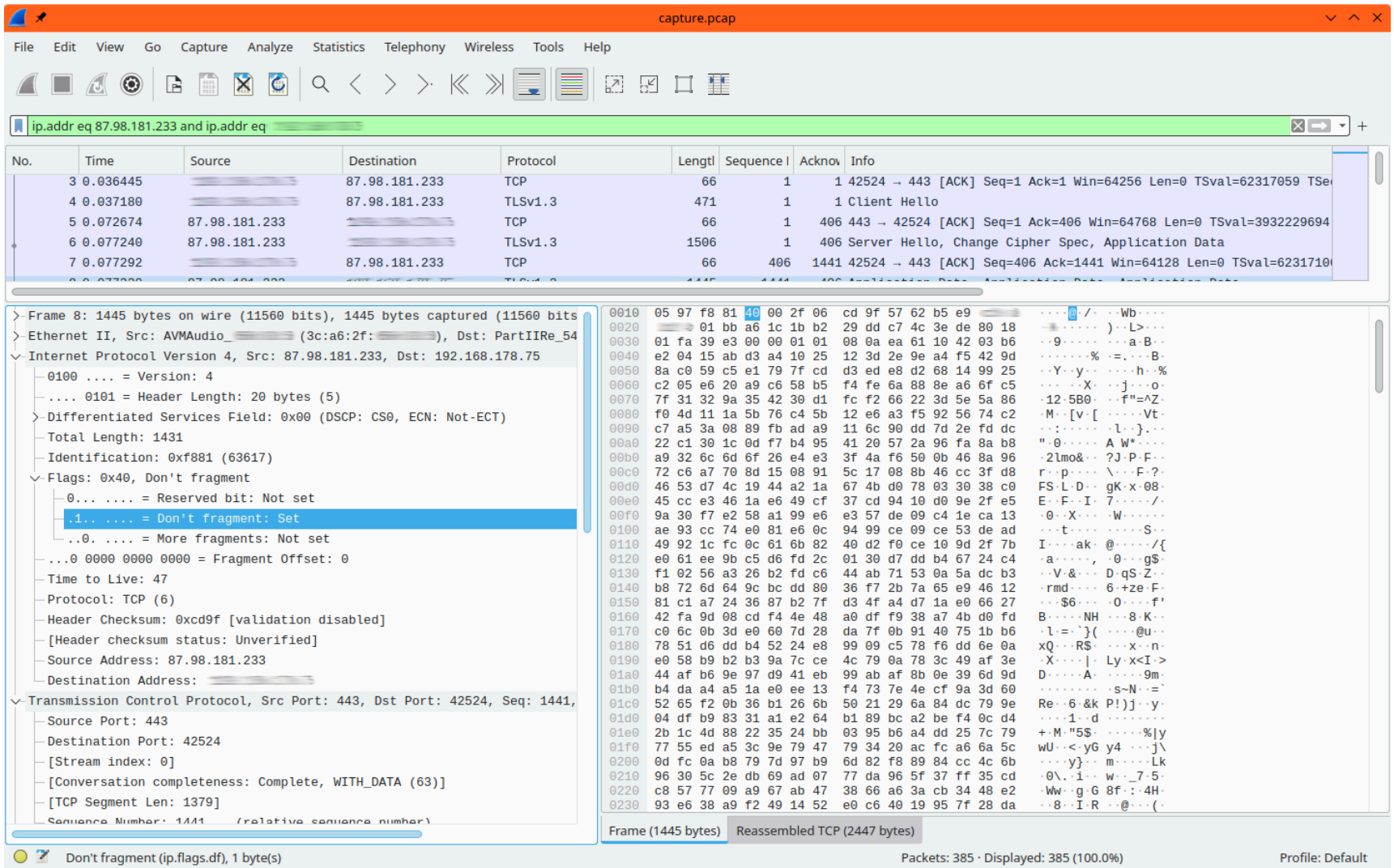
```
# tcpdump -i eth0
18:41:22.913058 IP localhost.localnet.40764 > srv.localnet: 14324+ AAAA? bootlin.com. (29)
18:41:22.913797 IP srv.localnet > localhost.localnet.40764: 14324 0/1/0 (89)
18:41:22.914268 IP localhost.localnet > bootlin.com: ICMP echo request, id 3, seq 1, length 64
18:41:23.933063 IP localhost.localnet > bootlin.com: ICMP echo request, id 3, seq 2, length 64
18:41:24.957027 IP localhost.localnet > bootlin.com: ICMP echo request, id 3, seq 3, length 64
18:41:24.996415 IP bootlin.com > localhost.localnet: ICMP echo reply, id 3, seq 3, length 64
^C
# tcpdump -i eth0 tcp and not port 22
... IP B.https > A.38910: Flags [.], ack 469, win 501, options [...], length 0
... IP B.https > A.38910: Flags [P.], seq 2602:2857, ack 469, win 501, options [...], length 255
... IP A.38910 > B.https: Flags [.], ack 2857, win 501, options [...], length 0
... IP A.38910 > B.https: Flags [P.], seq 469:621, ack 2857, win 501, options [...], length 152
... IP B.https > A.38910: Flags [.], ack 621, win 501, options [...], length 0
... IP B.https > A.38910: Flags [P.], seq 2857:3825, ack 621, win 501, options [...], length 968
... IP A.38910 > B.https: Flags [P.], seq 621:779, ack 3825, win 501, options [...], length 158
^C
#
```



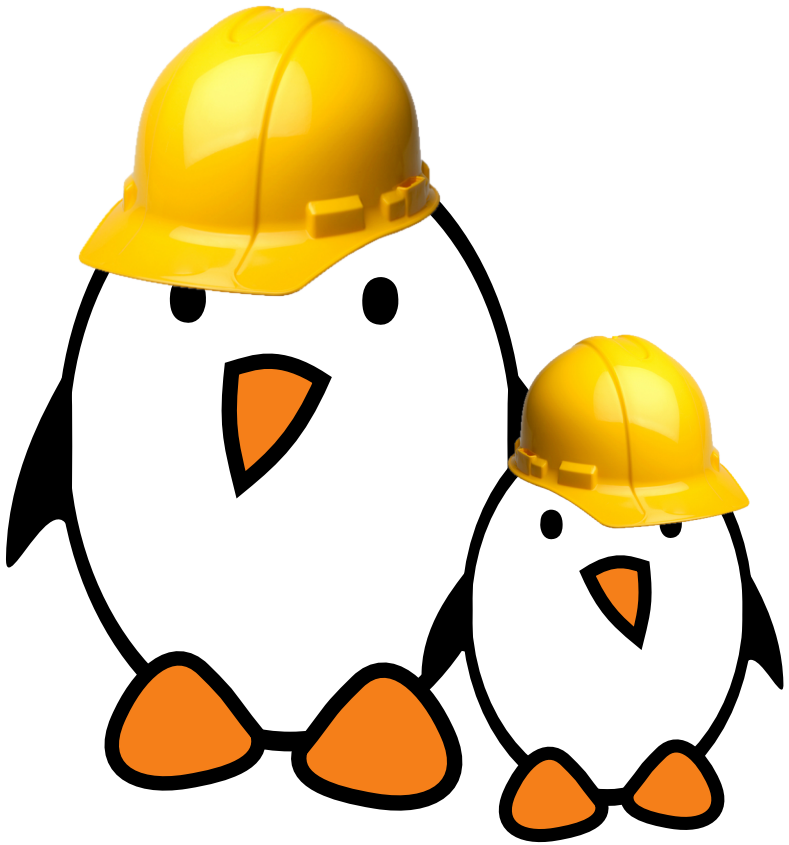
Wireshark

- Similar to tcpdump, but with a GUI
- Also based on libpcap
  - Can capture and use the same BPF capture filters
  - Can load and save the same file formats
    - Useful for embedded: capture on the target with tcpdump, analyze on the host with Wireshark
- Has *dissectors* to decode hundreds of protocols
  - Each individual value from each packet is dissected into a separate field
  - Fields are very fine-grained, at least for the most common protocols
- Has *display filters* that allow filtering *already captured* packets
  - Each dissected field is also a filter key
- Can also capture and decode Bluetooth, USB, D-Bus and more

Wireshark


capture.pcap
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
ip.addr eq 87.98.181.233 and ip.addr eq
No. Time Source Destination Protocol Lengtl Sequence I Acknov Info
3 0.036445 87.98.181.233 TCP 66 1 1 42524 → 443 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=62317059 TSe
4 0.037180 87.98.181.233 TLSv1.3 471 1 1 Client Hello
5 0.072674 87.98.181.233 TCP 66 1 406 443 → 42524 [ACK] Seq=1 Ack=406 Win=64768 Len=0 TSval=3932229694
6 0.077240 87.98.181.233 TLSv1.3 1506 1 406 Server Hello, Change Cipher Spec, Application Data
7 0.077292 87.98.181.233 TCP 66 406 1441 42524 → 443 [ACK] Seq=406 Ack=1441 Win=64128 Len=0 TSval=6231710
Frame 8: 1445 bytes on wire (11560 bits), 1445 bytes captured (11560 bits
Ethernet II, Src: AVMAudio_ (3c:a6:2f:), Dst: PartIIRe_54
Internet Protocol Version 4, Src: 87.98.181.233, Dst: 192.168.178.75
0100 .... = Version: 4
.... 0101 = Header Length: 20 bytes (5)
Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
Total Length: 1431
Identification: 0xf881 (63617)
Flags: 0x40, Don't fragment
0... .... = Reserved bit: Not set
.1.. .... = Don't fragment: Set
..0. .... = More fragments: Not set
...0 0000 0000 0000 = Fragment Offset: 0
Time to Live: 47
Protocol: TCP (6)
Header Checksum: 0xcd9f [validation disabled]
[Header checksum status: Unverified]
Source Address: 87.98.181.233
Destination Address:
Transmission Control Protocol, Src Port: 443, Dst Port: 42524, Seq: 1441,
Source Port: 443
Destination Port: 42524
[Stream index: 0]
[Conversation completeness: Complete, WITH_DATA (63)]
[TCP Segment Len: 1379]
0010 05 97 f8 81 40 00 2f 06 cd 9f 57 62 b5 e9
0020 01 bb a6 1c 1b b2 29 dd c7 4c 3e de 80 18
0030 01 fa 39 e3 00 00 01 01 08 0a ea 61 10 42 03 b6
0040 e2 04 15 ab d3 a4 10 25 12 3d 2e 9e a4 f5 42 9d
0050 8a c0 59 c5 e1 79 7f cd d3 ed e8 d2 68 14 99 25
0060 c2 05 e6 20 a9 c6 58 b5 f4 fe 6a 88 8e a6 6f c5
0070 7f 31 32 9a 35 42 30 d1 fc f2 66 22 3d 5e 5a 86
0080 f0 4d 11 1a 5b 76 c4 5b 12 e6 a3 f5 92 56 74 c2
0090 c7 a5 3a 08 89 fb ad a9 11 6c 90 dd 7d 2e fd dc
00a0 22 c1 30 1c 0d f7 b4 95 41 20 57 2a 96 fa 8a b8
00b0 a9 32 6c 6d 6f 26 e4 e3 3f 4a f6 50 0b 46 8a 96
00c0 72 c6 a7 70 8d 15 08 91 5c 17 08 8b 46 cc 3f d8
00d0 46 53 d7 4c 19 44 a2 1a 67 4b d0 78 03 30 38 c0
00e0 45 cc e3 46 1a e6 49 cf 37 cd 94 10 d0 9e 2f e5
00f0 9a 30 f7 e2 58 a1 99 e6 e3 57 de 09 c4 1e ca 13
0100 ae 93 cc 74 e0 81 e6 0c 94 99 ce 09 ce 53 de ad
0110 49 92 1c fc 0c 61 6b 82 40 d2 f0 ce 10 9d 2f 7b
0120 e0 61 ee 9b c5 d6 fd 2c 01 30 d7 dd b4 67 24 c4
0130 f1 02 56 a3 26 b2 fd c6 44 ab 71 53 0a 5a dc b3
0140 b8 72 6d 64 9c bc dd 80 36 f7 2b 7a 65 e9 46 12
0150 81 c1 a7 24 36 87 b2 7f d3 4f a4 d7 1a e0 66 27
0160 42 fa 9d 08 cd f4 4e 48 a0 df f9 38 a7 4b d0 fd
0170 c0 6c 0b 3d e0 60 7d 28 da 7f 0b 91 40 75 1b b6
0180 78 51 d6 dd b4 52 24 e8 99 09 c5 78 f6 dd 6e 0a
0190 e0 58 b9 b2 b3 9a 7c ce 4c 79 0a 78 3c 49 af 3e
01a0 44 af b6 9e 97 d9 41 eb 99 ab af 8b 0e 39 6d 9d
01b0 b4 da a4 a5 1a e0 ee 13 f4 73 7e 4e cf 9a 3d 60
01c0 52 65 f2 0b 36 b1 26 6b 50 21 29 6a 84 dc 79 9e
01d0 04 df b9 83 31 a1 e2 64 b1 89 bc a2 be f4 0c d4
01e0 2b 1c 4d 88 22 35 24 bb 03 95 b6 a4 dd 25 7c 79
01f0 77 55 ed a5 3c 9e 79 47 79 34 20 ac fc a6 6a 5c
0200 0d fc 0a b8 79 7d 97 b9 6d 82 f8 89 84 cc 4c 6b
0210 96 30 5c 2e db 69 ad 07 77 da 96 5f 37 ff 35 cd
0220 c8 57 77 09 a9 67 ab 47 38 66 a6 3a cb 34 48 e2
0230 93 e6 38 a9 f2 49 14 52 e0 c6 40 19 95 7f 28 da
Frame (1445 bytes)
Reassembled TCP (2447 bytes)
Don't fragment (ip.flags.df), 1 byte(s)
Packets: 385 · Displayed: 385 (100.0%)
Profile: Default

Practical lab - System Status



Check what is running on a system and its load

- Observe processes and IOs
- Display memory mappings
- Monitor resources